Web UI Design

Codebase Q&A and PR Review Assistant, Phase 4

Author	Todd Oppel
Repository	github.com/tpotechno/codebase-qa-assistant
Version	0.4.0 (Phase 4 complete)
Date	October 20, 2026
Status	Merged in PR #2; 64 server tests and 13 Angular tests passing

PDF version: web_ui_design.pdf · Earlier phases: mcp_server_design.md, pr_review_design.md

1. Summary

Phase 4 adds a browser interface for both things the assistant does. **Ask a question** streams the answer as it is written, and a Steps panel shows every tool call live: what was searched, which lines were read, and each result one click away. **Review a pull request** shows the verdict, the summary, and comments grouped by file with their line numbers. A **Post to GitHub** button publishes exactly the draft on screen.

Behind the UI is a small HTTP API (`src/web-app.ts`) that streams Server-Sent Events from the two agents built in earlier phases. The front end is an Angular 21 app (`web/`) built with standalone components and signals, with no zone.js. The server runs only on the user's own machine, because it holds the Anthropic and GitHub credentials, and it is hardened against use from other websites.

2. Why

The command line proved the agents work, but it hides what makes them interesting: watching an agent decide what to search, read the code, and build an answer from evidence. A visual trace makes that process visible. That matters for trust, because the user can see what the answer is based on, and for demos, because the agent's work is the story.

The UI also fixed a real limitation from Phase 3. `review --post` ran a fresh review, so what got posted could differ from what the user had read. In the browser, the user reads the review and then posts that exact draft.

Goals

- Stream answers and agent steps in real time, in both modes.
- Post exactly the review the user read, at most once, and never after the PR has changed.
- Keep the stored credentials safe from other websites the user visits.
- Stop spending when the user stops caring: Stop button, closed tab, or a stuck run.
- Work well at desktop and phone widths, in light and dark themes, and with a keyboard.
- Let anyone try the UI without an API key.

Non-goals (this phase)

- Hosting the app for other people, user accounts, or OAuth (Phase 5).
- Conversation memory: each question stands alone.
- Choosing a repository from the UI; it is set by `REPO_PATH`.
- Measuring answer or review quality (Phase 5).

3. What was built

Architecture

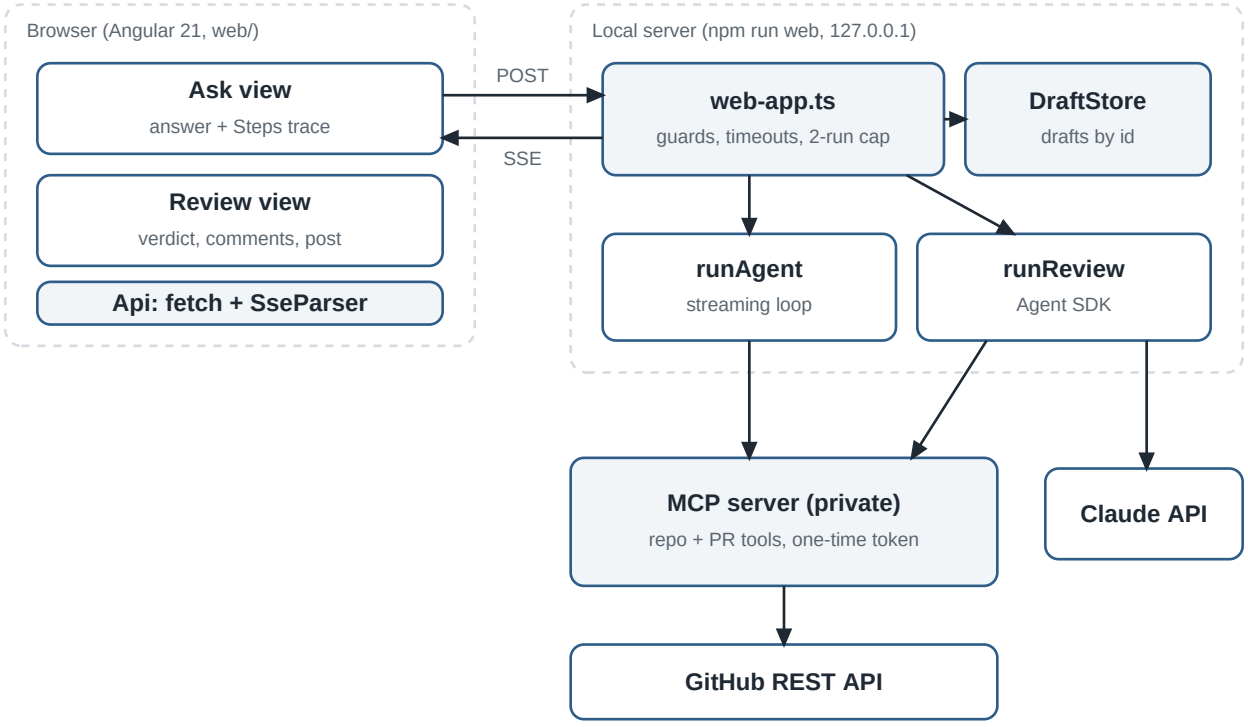

Figure 1. Runtime architecture. Dashed outlines group what runs together.

Server components

File	Responsibility
`src/web-app.ts`	The API: request guards, the three POST endpoints, Server-Sent Events, the draft store, timeouts, the concurrency cap, and serving the built Angular app. Created from injected dependencies, so tests swap in fake Claude, GitHub, and agent streams.
`src/web-server.ts`	Entry point for `npm run web`: reads `.env`, validates the port and budget, starts a private MCP server with a one-time token, and starts the API.
`src/agent.ts`	The hand-written loop gained optional streaming (`onText` with a turn number), `onToolResult` with matching ids, and an `AbortSignal`.
`src/review.ts`	The review agent accepts an `AbortController`, which is passed through to the Agent SDK.

File	Responsibility
`scripts/demo-server.ts`	The UI with a scripted stand-in for Claude: real repo tools, a sample PR from an in-memory GitHub, and no API key or spend.

Angular components

File	Responsibility
`app/app.ts`	Shell: header with the repo name and the mode switch, loads `/api/config`. Both views stay mounted, so switching modes keeps an answer or a review.
`app/ask/`	Question box with example questions, the streamed answer rendered from Markdown, and the trace. Moves Claude's pre-tool notes out of the answer and into the trace.
`app/review/`	PR input, verdict and summary, comments grouped by file, and the post flow: confirm, posting, posted with a link.
`app/trace/`	The step list both views share, styled like a diff's line-number gutter.
`app/api.ts`, `app/sse.ts`	`fetch`-based streaming client and an incremental Server-Sent Events parser.
`app/markdown.pipe.ts`	Markdown to HTML for model output, with raw HTML escaped.

API

Endpoint	JSON body	Response
`GET /api/config`	—	Repo name and path, model, whether posting is available, review budget
`POST /api/ask`	`question`	Event stream: `text` (delta, turn), `tool_call` (id, name, input), `tool_result` (id, isError, preview), `done` (answer, turns, tool calls, tokens), or `error`
`POST /api/review`	`pr`	Event stream: `started` (PR title, link, head commit, file count), `tool_call`, `done` (review, draftId, cost, turns, duration), or `error`
`POST /api/review/post`	`draftId`	Posted review link and comment counts; 404 if the draft is gone, 409 if already posted or the PR has new commits

4. How it works

Asking a question

1. The browser POSTs the question and reads the response as an event stream.
2. The server opens a tool connection to its private MCP server, so Ask gets the repo tools and the PR tools, and runs the hand-written agent loop with streaming on.
3. Each text delta is sent with its turn number. Each tool call and result is sent with a shared id, so the UI marks the right step done or failed.
4. In the browser, text that arrives before a tool call is a working note ("I'll find where…"), so it moves into the trace and the answer area clears. The final turn's text is the answer.

5. `done` carries the authoritative answer and the turn, tool-call, and token counts.

Reviewing a pull request

1. The server fetches the PR fresh, bypassing the 60-second cache. This validates the PR and pins the commit the review is based on.
2. `started` gives the UI the title, link, and head commit right away.
3. The Agent SDK review runs as in Phase 3, read-only and budget-capped. Each tool call is streamed. The review agent reports calls but not results, so a new call marks the previous one done.
4. When the review finishes, the server stores it in the `DraftStore` and sends the review with a `draftId`.

Posting a draft

The browser sends only the `draftId`. The server looks up the draft it generated, marks it as being posted before doing any other work (so a second request at the same moment sees it as in progress), and calls `postReview` with the pinned commit. Outcomes:

- **Posted:** the draft is marked posted and the link is returned. Any repeat gets 409 "already posted" with the same link.
- **PR has new commits:** a typed `StaleHeadError` becomes 409, and the draft returns to ready.
- **Other failure:** 502, and the draft returns to ready, so the user can retry.

Drafts live for an hour. At most 20 are kept, oldest evicted first, but a draft that is mid-post is never evicted.

Streaming

`EventSource` only supports GET, and these endpoints are POSTs with JSON bodies, so the browser uses `fetch` and reads the response body through a small parser (`SseParser`). The parser buffers partial frames across network chunks and normalizes line endings across the whole buffer. Its unit tests caught a real bug here: a `\r\n` pair split across two chunks stalled the parser.

Cancellation and timeouts

Every run gets an `AbortController` that fires when:

- the browser disconnects (Stop button or a closed tab), or
- the run passes 10 minutes.

The signal is passed into the Claude API calls and the Agent SDK. The server also wraps each run in `untilAborted`, which settles the request as soon as the signal fires, even if the work underneath ignores it (for example, a stuck tool). The response closes and the concurrency slot frees either way. A timed-out run tells the browser "Stopped: no result after 10 minutes." At most two runs can be active at once; a third gets 429.

5. Interface design

The UI's job is to show what the agent is doing while it works, so the trace got the visual emphasis and everything around it is kept quiet.

- **The trace is styled like a diff gutter.** Each tool call gets a number in a tinted gutter column. Numbering is honest here because the calls really are a sequence. The running step's gutter takes the accent color, so the eye finds it at once. Review comments use the same gutter for their line numbers, echoing the line-numbered diffs the reviewer reads.
- **Color carries meaning.** One accent, teal (`#0E7468`), marks actions and the running step. Red, amber, and gray only ever mean blocker, suggestion, and nit. The paper background is cool gray and the text is navy ink; the dark theme keeps the same roles.
- **Type.** IBM Plex Sans for the interface, IBM Plex Mono only for real code: paths, tool inputs, line numbers. The fonts are bundled with the app, so there are no third-party requests and it works offline.
- **Layout.** A left-aligned reading column for the question and answer, with the trace in a sticky right column on wide screens. Below 56rem the trace moves above the answer and can be collapsed.
- **Writing.** Buttons say what happens ("Post to GitHub", then "Post review" to confirm). The post button explains its scope ("Posts exactly this review, as a comment. It never approves or blocks the PR."). Errors say what went wrong and what to do, and the empty state offers example questions.
- **Accessibility.** The answer area is `aria-live`, the trace sets `aria-busy` while running, status glyphs have text labels, focus is visible everywhere, and the spinner and streaming caret stop for `prefers-reduced-motion`.

6. Security

Risk	Mitigation
A malicious website calls the local API (spending money, posting under the user's name)	Listens on `127.0.0.1` only. Requests whose `Host` isn't the server's own are rejected (DNS rebinding). Non-GET requests must be JSON (a cross-site form can't send JSON without a CORS preflight, which the server never approves) and a foreign `Origin` is rejected.
Something that reaches the API posts arbitrary text to GitHub	The server posts only drafts it generated, by id. Review text sent by a client is rejected.
Double-clicks or retries post twice	A draft is claimed before any other work and can be posted once; the button also disables while posting.
Comments landing on the wrong lines	Reviews start from the PR's fresh head commit; posting refuses if the PR has moved since.
HTML or script in model output (which can quote PR content)	Raw HTML is escaped when rendering Markdown, and Angular's `[innerHTML]` sanitizer is a second layer.
Runaway cost	Stop and tab-close cancel runs, a 10-minute timeout, a two-run cap, and the per-review spend cap from Phase 3.
Credentials leaking to the browser	The API key and GitHub token stay in the server process; the browser never sees them. The private MCP server uses a one-time random token.

7. Key decisions and alternatives

Decision	Alternative considered	Why this choice
Server-Sent Events over `fetch`	WebSockets; `EventSource`	One-way streaming is all that's needed; plain HTTP is simpler to secure and test. `EventSource` can't send a POST body.
Server keeps drafts and posts by id	Browser sends the review back to post	Makes "posts exactly the draft on screen" true by construction, blocks forged posts, and allows exactly-once posting.
Angular 21	Angular 22 (latest)	Angular 22 requires Node 24.15 or newer; 21 runs on the Node already installed and matches recent professional work. Upgrading later is small.
Standalone components, signals, zoneless	NgModules with zone.js	The current Angular model: less boilerplate and predictable change detection, and streaming updates map naturally onto signals.
Both views stay mounted	Router with separate pages	Switching modes keeps a long answer or a review; two views don't need routing.
Fonts bundled with `@fontsource`	Google Fonts	Works offline, no third-party requests from a tool that handles code.
Scripted demo server	Recorded screenshots or video only	A live, clickable UI with real repo tools and no API key, for demos and for UI development.
`untilAborted` around runs	Rely on each agent honoring the signal	A stuck tool can ignore cancellation; the server must free the slot regardless. Found when the first timeout test hung.

8. Review and testing history

PR #2 was reviewed twice by the assistant's own reviewer, and every finding was checked against the code before it was fixed.

Round 1, 7 findings, all fixed:

1. Posting had no guard against double-posting. Drafts can now be posted at most once.
2. Loading the PR wasn't cancellable. The review now stops if the browser leaves.
3. The 409 depended on the wording of an error message. Now a typed `StaleHeadError`.
4. The post endpoint accepted any review text. The server now posts only its own drafts, by id. This became the central design of the post flow.
5. An invalid `WEB_PORT` wasn't caught. Now validated.
6. The streaming fallback and abort behavior were undocumented. Comments added.
7. Model output relied on one HTML defense. Raw HTML is now escaped too.

Round 2, 5 findings, all fixed:

1. A stuck run could hold a slot forever. Added the 10-minute timeout and `untilAborted`.
2. Eviction could drop a draft mid-post. Drafts being posted are now skipped.
3. A slot was released before its tool connection closed. Reordered.
4. **Bug:** a review could pin a cached, outdated commit, so posting was wrongly refused. Reviews now fetch the PR fresh.

5. Aborts threw a generic error. They now raise a named `AbortError`.

Found by our own testing before review:

- The split-`\r\n` stall in the event parser (unit tests).
- On phones, the collapse arrows meant for the Steps heading appeared on every result toggle, and the streaming caret dropped below lists (browser screenshots).
- Google Fonts was blocked in the test environment, which led to bundling the fonts instead.

Reviewer limitation observed: on this ~4,000-line PR, the reviewer skipped the Angular components and tests. It said so honestly in its summary, but nothing measured it. Coverage reporting is the first next step (Section 11).

9. Testing

Area	What is verified
Web API (server)	Real event streams end to end with a fake streaming Claude client and fake GitHub; posting once, two simultaneous posts publishing once, forged review text rejected, stale commits, missing tokens, draft expiry and eviction, a stalled run timing out and freeing its slot, fresh PR fetches on back-to-back reviews, foreign Host/Origin and non-JSON rejected, port validation
Agent loop	Streaming text by turn, tool results matched by id, abort raises `AbortError`
Angular	The event parser (partial frames, split CRLF), tool summaries, the trace (numbering, status, notes), review helpers (grouping, counts, posted message), and Markdown escaping
In a browser	Both views at desktop and phone width, light and dark; the full post flow, including a double-click; no console errors. Run against the demo server with Playwright.

CI runs two jobs on every push: the server suite on Node 22, and the Angular tests and production build on Node 24. The production bundle is about 70 kB compressed.

10. Configuration and usage

Variable	Default	Purpose
`WEB_PORT`	`3334`	Port for `npm run web`
`REPO_PATH`	current folder	Repository that Ask questions are about
`GITHUB_TOKEN`	unset	Needed to post reviews and for private repos
`REVIEW_MAX_BUDGET_USD`	`1`	Spend cap per review

```
npm run web:install   # once
npm run web:build
npm run web           # http://localhost:3334

npm run web:demo      # scripted, no API key: http://localhost:3335
npm run web:dev       # with npm run web running: live-reload UI on :4200
npm run web:test      # Angular unit tests
```

11. Limitations and next steps

- **Review coverage isn't measured.** The reviewer can skip parts of a large diff. Next: track which diff pages and files it read, and show "Read 3 of 6 pages" with a warning in the UI and the command line.
- **No conversation memory.** Each question stands alone. Next: send prior turns with follow-up questions.
- **One repository per server**, set by `REPO_PATH`.
- **Drafts live in memory.** Restarting the server loses unposted drafts. Fine for a local tool; a hosted version would need storage.
- **Local and single-user by design.** Hosting it for others needs authentication, per-user credentials, and rate limits (Phase 5).
- **No browser tests in CI.** The Playwright checks were run by hand against the demo server; adding them to CI is straightforward.
- **Phase 5:** an evaluation suite for answer and review quality, cost tracking and tracing, OAuth, and a hosted demo.